Modul 295 - Dokumentation

Autor: Julian Schiller

Datum: 24.04.2026

Inhaltsverzeichnis

1. Libraries und Frameworks
2. API Analyse
3. API Definition
4. REST API Dokumentation
5. Testfälle
    - 5.1 Adressen
    - 5.2 Medien
    - 5.3 Kunden
    - 5.4 Ausleihen

1. Libraries und Frameworks

Library	Version	Zweck
Spring Boot	4.0.5	Application Framework
Spring Data JPA	-	DB-Zugriff
MySQL Connector	-	JDBC Driver
Lombok	-	Reduktion von Boilerplate
springdoc-openapi	2.8.4	API-Dokumentation via Swagger UI

Als IDE wurde VS Code verwendet und als Programmiersprache Java 21. Als Build Tool wurde Gradle 8.14.4 verwendet und Groovy als DSL. Die History wurde mit Git gemacht, welches ebenfalls direkt mit einem Github Repo verbunden wurde um einen zweiten Speicherort zu haben.

2. API Analyse

Wer	Anforderung	HTTP-Methode
Der Benutzer	kann alle Medienobjekte auflisten	GET
Der Benutzer	kann Medien nach Titel suchen	GET
Der Benutzer	kann ein Medienobjekt anhand der ID abrufen	GET
Der Benutzer	kann ein neues Medienobjekt erstellen	POST
Der Benutzer	kann ein bestehendes Medienobjekt aktualisieren	PATCH
Der Benutzer	kann ein Medienobjekt löschen	DELETE
Der Benutzer	kann alle Adressen auflisten	GET
Der Benutzer	kann Adressen nach Strasse und Hausnummer suchen	GET
Der Benutzer	kann Adressen nach Postleitzahl suchen	GET
Der Benutzer	kann eine neue Adresse erstellen	POST
Der Benutzer	kann eine Adresse löschen (nur wenn sie nicht referenziert wird)	DELETE
Der Benutzer	kann einen Kunden anhand der ID abrufen	GET
Der Benutzer	kann Kunden nach Namen suchen	GET
Der Benutzer	kann Kunden nach Adress-ID suchen	GET
Der Benutzer	kann einen neuen Kunden erstellen	POST
Der Benutzer	kann einen bestehenden Kunden aktualisieren	PATCH
Der Benutzer	kann einen Kunden löschen	DELETE
Der Benutzer	kann alle Ausleihen auflisten	GET
Der Benutzer	kann eine neue Ausleihe erstellen	POST
Der Benutzer	kann eine bestehende Ausleihe verlängern	PATCH
Der Benutzer	kann eine Ausleihe beenden (über mediaId)	DELETE

3. API Definition

In diesem Abschnitt wird die gesamte API Definition aufgezeigt, es wird gezeigt welche URL's es gibt, der jeweilige Anwendungsfall und der Request Body.

Medien

URL	Methode	Anwendungsfall	Request Body
/library/media	GET	Alle Medien abrufen	-
/library/media?title=	GET	Alle Medien mit bestimmtem Titel abrufen	-
/library/media/{id}	GET	Ein Medium mit bestimmter ID abrufen	-
/library/media	POST	Ein neues Medium erstellen	Das neue Medium als JSON
/library/media/{id}	PATCH	Ein bestehendes Medium bearbeiten	Das bearbeitete Medium als JSON
/library/media/{id}	DELETE	Ein Medium löschen	-

JSON-Struktur POST

```
{
    "title":"",
    "author":"",
    "genre":"",
    "minage":12,
    "isbn":"",
    "locationcode":""
}
```

Titel und Autor müssen gesendet werden, alles andere ist optional.

JSON-Struktur PATCH

```
{
    "genre":"",
    "minage":12,
    "isbn":"",
    "locationcode":""
}
```

Alle Attribute sind optional, aber nur diese dürfen bearbeitet werden.

Adresse

URL	Methode	Anwendungsfall	Request Body
/library/addresses	GET	Alle Adressen abrufen	-
/library/addresses?address=	GET	Alle Adressen mit bestimmter Strasse und Hausnummer abrufen	-
/library/addresses?zip=	GET	Alle Adressen mit bestimmter Postleitzahl abrufen	-
/library/addresses	POST	Eine neue Adresse erstellen	Die neue Adresse als JSON
/library/addresses/{id}	DELETE	Eine Adresse löschen (schlägt fehl, wenn sie noch von einem Kunden referenziert wird)	-

JSON-Struktur POST

```
{
    "address":"",
    "city":"",
    "zip":""
}
```

Alle Attribute müssen gesendet werden.

Kunde

URL	Methode	Anwendungsfall	Request Body
/library/customers/{id}	GET	Kunden mit bestimmter ID abrufen	-
/library/customers?name=	GET	Alle Kunden mit bestimmtem Namen abrufen	-
/library/customers?addressId=	GET	Alle Kunden mit bestimmter Adresse abrufen	-
/library/customers	POST	Einen neuen Kunden erstellen	Der neue Kunde als JSON
/library/customers/{id}	PATCH	Einen bestehenden Kunden bearbeiten	Der bearbeitete Kunde als JSON
/library/customers/{id}	DELETE	Einen Kunden löschen	-

JSON-Struktur POST

```
{
    "firstname":"",
    "lastname":"",
    "birthdate":"YYYY-MM-DD",
    "email":"",
    "address":{
        "address":"",
        "city":"",
        "zip":""
    }
}
```

Alle Attribute müssen gesendet werden.

JSON-Struktur PATCH

```
{
    "email":"",
    "address":{
        "address":"",
        "city":"",
        "zip":""
    }
}
```

Alle Attribute sind optional, aber nur diese dürfen bearbeitet werden.

Ausleihe

URL	Methode	Anwendungsfall	Request Body
/library/borrowings	GET	Alle Ausleihen abrufen	-
/library/borrowings	POST	Eine neue Ausleihe erstellen	Die neue Ausleihe als JSON
/library/borrowings/{id}	PATCH	Eine bestehende Ausleihe verlängern	Die bearbeitete Ausleihe als JSON
/library/borrowings?mediaId=	DELETE	Eine Ausleihe beenden	-

JSON-Struktur POST

```
{
    "duration":14,
    "customer":{
        "id":3
    },
    "media":{
        "id":12
    }
}
```

Dauer ist optional (hat einen Standardwert), Kunde und Medium müssen gesendet werden.

JSON-Struktur PATCH

```
{
    "duration":28
}
```

Dauer ist das einzige Attribut, das bearbeitet werden kann.

4. REST API Dokumentation

Die API Dokumentation kann im zip File mit dem Namen api-docs.json gefunden werden.

5. Testfälle

Die Tests wurden mit der VS Code REST Client Extension manuell ausgeführt. Jeder Test zeigt den gesendeten Request und die erhaltene Response.

5.1 Adressen

Alle Adressen abrufen

Request: `GET /library/addresses`

Erwartet: 200 OK, JSON Array mit allen Adressen

Response(20ms)

```
HTTP/1.1 200
Content-Type: application/json
Content-Length: 300
Date: Mon, 20 Apr 2026 11:19:46 GMT
Connection: close

[
  {
    "id": 2,
    "address": "Hochwalderstrasse 23",
    "city": "Roeschenz",
    "zip": "4095"
  },
  {
    "id": 3,
    "address": "Hochwalderstrasse 23",
    "city": "Roeschenz",
    "zip": "4095"
  },
  {
    "id": 4,
    "address": "Hochwalderstrasse 123",
    "city": "Roeschenz",
    "zip": "4095"
  },
  {
    "id": 5,
    "address": "Hochwalderstrasse 23",
    "city": "Anderer Ort",
    "zip": "4090"
  }
]
```

Adressen nach Postleitzahl filtern

Request: `GET /library/addresses?zip=4090`

Erwartet: 200 OK, JSON Array mit Adressen mit PLZ 4090

Response(8ms)

```
HTTP/1.1 200
Content-Type: application/json
Content-Length: 77
Date: Mon, 20 Apr 2026 11:21:21 GMT
Connection: close

[
  {
    "id": 5,
    "address": "Hochwalderstrasse 23",
    "city": "Anderer Ort",
    "zip": "4090"
  }
]
```

Adressen nach Strasse filtern

Request: `GET /library/addresses?address=Hochwalderstrasse 23`

Erwartet: 200 OK, JSON Array mit passenden Adressen

Response(7ms)

```
HTTP/1.1 200
Content-Type: application/json
Content-Length: 225
Date: Mon, 20 Apr 2026 11:21:52 GMT
Connection: close

[
  {
    "id": 2,
    "address": "Hochwalderstrasse 23",
    "city": "Roeschenz",
    "zip": "4095"
  },
  {
    "id": 3,
    "address": "Hochwalderstrasse 23",
    "city": "Roeschenz",
    "zip": "4095"
  },
  {
    "id": 5,
    "address": "Hochwalderstrasse 23",
    "city": "Anderer Ort",
    "zip": "4090"
  }
]
```

Neue Adresse erstellen

Request: `POST /library/addresses`

Erwartet: 201 Created, Adresse wird in der Datenbank gespeichert

Response(240ms)

```
HTTP/1.1 201
Content-Type: application/json
Content-Length: 75
Date: Mon, 20 Apr 2026 11:23:00 GMT
Connection: close

{
  "id": 8,
  "address": "Hochwalderstrasse 23",
  "city": "Anderer Ort",
  "zip": "4090"
}
```

Adresse löschen

Request: `DELETE /library/addresses/2`

Erwartet: 204 No Content, Adresse wird gelöscht

Response(369ms)

```
HTTP/1.1 204
Date: Mon, 20 Apr 2026 11:23:48 GMT
Connection: close
```

Verifikation: GET nach dem Löschen

Response(28ms) × 05-delete-address.png U

```
HTTP/1.1 200
Content-Type: application/json
Content-Length: 378
Date: Mon, 20 Apr 2026 11:24:02 GMT
Connection: close

[
  {
    "id": 3,
    "address": "Hochwalderstrasse 23",
    "city": "Roeschenz",
    "zip": "4095"
  },
  {
    "id": 4,
    "address": "Hochwalderstrasse 123",
    "city": "Roeschenz",
    "zip": "4095"
  },
  {
    "id": 5,
    "address": "Hochwalderstrasse 23",
    "city": "Anderer Ort",
    "zip": "4090"
  },
  {
    "id": 7,
    "address": "Hochwalderstrasse 23",
    "city": "Anderer Ort",
    "zip": "4090"
  },
  {
    "id": 8,
    "address": "Hochwalderstrasse 23",
    "city": "Anderer Ort",
    "zip": "4090"
  }
]
```

Referenzierte Adresse löschen

Request: `DELETE /library/addresses/4`

Erwartet: 400 Bad Request, Adresse wird noch von einem Kunden referenziert

Response(24ms)

```
HTTP/1.1 400
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 20 Apr 2026 11:24:31 GMT
Connection: close

{
  "timestamp": "2026-04-20T11:24:31.350Z",
  "status": 400,
  "error": "Bad Request",
  "path": "/library/addresses/4"
}
```

Nicht existierende Adresse löschen

Request: `DELETE /library/addresses/9999`

Erwartet: 404 Not Found

Response(8ms)

```
HTTP/1.1 404
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 20 Apr 2026 11:25:02 GMT
Connection: close

{
  "timestamp": "2026-04-20T11:25:02.398Z",
  "status": 404,
  "error": "Not Found",
  "path": "/library/addresses/9999"
}
```

5.2 Medien

Alle Medien abrufen

Request: `GET /library/media`

Erwartet: 200 OK, JSON Array mit allen Medien

Response(9ms)

```
HTTP/1.1 200
Content-Type: application/json
Content-Length: 269
Date: Mon, 20 Apr 2026 11:25:23 GMT
Connection: close

[
  {
    "id": 2,
    "title": "Harry Potter",
    "author": "J.K. Rowling",
    "genre": "Adventure",
    "minage": 10,
    "isbn": "978-3-16-148410-0",
    "locationcode": "A1"
  },
  {
    "id": 4,
    "title": "Harry Potter",
    "author": "J.K. Rowling",
    "genre": "Fantasy",
    "minage": 12,
    "isbn": "978-3-16-148410-0",
    "locationcode": "A1"
  }
]
```

Medien nach Titel filtern

Request: `GET /library/media?title=Harry Potter`

Erwartet: 200 OK, JSON Array mit Medien mit Titel "Harry Potter"

Response(11ms) × 08-get-all-medias.png U

```
HTTP/1.1 200
Content-Type: application/json
Content-Length: 269
Date: Mon, 20 Apr 2026 11:25:53 GMT
Connection: close

[
  {
    "id": 2,
    "title": "Harry Potter",
    "author": "J.K. Rowling",
    "genre": "Adventure",
    "minage": 10,
    "isbn": "978-3-16-148410-0",
    "locationcode": "A1"
  },
  {
    "id": 4,
    "title": "Harry Potter",
    "author": "J.K. Rowling",
    "genre": "Fantasy",
    "minage": 12,
    "isbn": "978-3-16-148410-0",
    "locationcode": "A1"
  }
]
```

Medium nach ID abrufen

Request: `GET /library/media/2`

Erwartet: 200 OK, JSON Objekt des Mediums

Response(10ms) ×

```
HTTP/1.1 200
Content-Type: application/json
Content-Length: 134
Date: Mon, 20 Apr 2026 11:26:29 GMT
Connection: close

{
  "id": 2,
  "title": "Harry Potter",
  "author": "J.K. Rowling",
  "genre": "Adventure",
  "minage": 10,
  "isbn": "978-3-16-148410-0",
  "locationcode": "A1"
}
```

Nicht existierendes Medium abrufen

Request: `GET /library/media/9999`

Erwartet: 404 Not Found

Response(7ms)

```
HTTP/1.1 404
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 20 Apr 2026 11:26:56 GMT
Connection: close

{
  "timestamp": "2026-04-20T11:26:56.052Z",
  "status": 404,
  "error": "Not Found",
  "path": "/library/media/9999"
}
```

Neues Medium erstellen

Request: `POST /library/media`

Erwartet: 201 Created, JSON Objekt des erstellten Mediums

Response(12ms)

```
HTTP/1.1 201
Content-Type: application/json
Content-Length: 133
Date: Mon, 20 Apr 2026 11:28:01 GMT
Connection: close

{
  "id": 5,
  "title": "Fourth Wing",
  "author": "Rebecca Yarros",
  "genre": "Fantasy",
  "minage": 14,
  "isbn": "978-3-16-148410-0",
  "locationcode": "A1"
}
```

Medium bearbeiten

Request: `PATCH /library/media/5`

Erwartet: 200 OK, Genre und Mindestalter werden aktualisiert

Response(13ms)

```
HTTP/1.1 200
Content-Type: application/json
Content-Length: 135
Date: Mon, 20 Apr 2026 11:28:38 GMT
Connection: close

{
  "id": 5,
  "title": "Fourth Wing",
  "author": "Rebecca Yarros",
  "genre": "Romantasy",
  "minage": 10,
  "isbn": "978-3-16-148410-0",
  "locationcode": "A1"
}
```

Nicht existierendes Medium bearbeiten

Request: `PATCH /library/media/9999`

Erwartet: 404 Not Found

Response(7ms)

```
HTTP/1.1 404
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 20 Apr 2026 11:29:08 GMT
Connection: close

{
  "timestamp": "2026-04-20T11:29:08.014Z",
  "status": 404,
  "error": "Not Found",
  "path": "/library/media/9999"
}
```

Medium löschen

Request: `DELETE /library/media/4`

Erwartet: 204 No Content, Medium wird gelöscht

Response(26ms)

```
HTTP/1.1 204
Date: Mon, 20 Apr 2026 11:51:31 GMT
Connection: close
```

Verifikation: GET nach dem Löschen

Response(17ms) × 18-get-customer-with-specific-id.png U 15-deleted-media.png U

```
HTTP/1.1 200
Content-Type: application/json
Content-Length: 272
Date: Mon, 20 Apr 2026 11:53:45 GMT
Connection: close

[
  {
    "id": 2,
    "title": "Harry Potter",
    "author": "J.K. Rowling",
    "genre": "Adventure",
    "minage": 10,
    "isbn": "978-3-16-148410-0",
    "locationcode": "A1"
  },
  {
    "id": 5,
    "title": "Fourth Wing",
    "author": "Rebecca Yarros",
    "genre": "Romantasy",
    "minage": 10,
    "isbn": "978-3-16-148410-0",
    "locationcode": "A1"
  }
]
```

Ausgeliehenes Medium löschen

Request: `DELETE /library/media/2`

Erwartet: 409 Conflict, Medium ist noch ausgeliehen

Response(11ms) × 15-deleted-media.png U

```
HTTP/1.1 409
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 20 Apr 2026 11:51:56 GMT
Connection: close

{
  "timestamp": "2026-04-20T11:51:56.090Z",
  "status": 409,
  "error": "Conflict",
  "path": "/library/media/2"
}
```

Nicht existierendes Medium löschen

Request: `DELETE /library/media/9999`

Erwartet: 404 Not Found

Response(7ms)

```
HTTP/1.1 404
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 20 Apr 2026 11:52:23 GMT
Connection: close

{
  "timestamp": "2026-04-20T11:52:23.306Z",
  "status": 404,
  "error": "Not Found",
  "path": "/library/media/9999"
}
```

5.3 Kunden

Kunde nach ID abrufen

Request: `GET /library/customers/1`

Erwartet: 200 OK, JSON Objekt des Kunden

Response(29ms)

```
HTTP/1.1 200
Content-Type: application/json
Content-Length: 195
Date: Mon, 20 Apr 2026 11:52:58 GMT
Connection: close

{
  "id": 1,
  "firstname": "Max",
  "lastname": "Meier",
  "birthdate": "1999-12-31T23:00:00.000Z",
  "email": "newmail@test.com",
  "address": {
    "id": 3,
    "address": "Hochwalderstrasse 23",
    "city": "Roeschenz",
    "zip": "4095"
  }
}
```

Nicht existierenden Kunden abrufen

Request: `GET /library/customers/9999`

Erwartet: 404 Not Found

Response(8ms)

```
HTTP/1.1 404
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 20 Apr 2026 11:54:22 GMT
Connection: close

{
  "timestamp": "2026-04-20T11:54:22.081Z",
  "status": 404,
  "error": "Not Found",
  "path": "/library/customers/9999"
}
```

Kunden nach Nachname filtern

Request: `GET /library/customers?name=Test`

Erwartet: 200 OK, JSON Array mit passenden Kunden

Response(9ms)

```
HTTP/1.1 200
Content-Type: application/json
Content-Length: 574
Date: Mon, 20 Apr 2026 11:58:38 GMT
Connection: close

[
  {
    "id": 2,
    "firstname": "Max",
    "lastname": "Test",
    "birthdate": "1999-12-31T23:00:00.000Z",
    "email": "max@meier.ch",
    "address": {
      "id": 3,
      "address": "Hochwalderstrasse 23",
      "city": "Roeschenz",
      "zip": "4095"
    }
  },
  {
    "id": 4,
    "firstname": "Max",
    "lastname": "Test",
    "birthdate": "1999-12-31T23:00:00.000Z",
    "email": "max@meier.ch",
    "address": {
      "id": 3,
      "address": "Hochwalderstrasse 23",
      "city": "Roeschenz",
      "zip": "4095"
    }
  },
  {
    "id": 5,
    "firstname": "Max",
    "lastname": "Test",
    "birthdate": "1999-12-31T23:00:00.000Z",
    "email": "max@meier.ch",
    "address": {
      "id": 3,
      "address": "Hochwalderstrasse 23",
      "city": "Roeschenz",
      "zip": "4095"
    }
  }
]
```

Kunden nach Adress-ID filtern

Request: `GET /library/customers?addressId=3`

Erwartet: 200 OK, JSON Array mit Kunden an dieser Adresse

Response(9ms)

```
HTTP/1.1 200
Content-Type: application/json
Content-Length: 770
Date: Mon, 20 Apr 2026 11:59:01 GMT
Connection: close

[
  {
    "id": 1,
    "firstname": "Max",
    "lastname": "Meier",
    "birthdate": "1999-12-31T23:00:00.000Z",
    "email": "newmail@test.com",
    "address": {
      "id": 3,
      "address": "Hochwalderstrasse 23",
      "city": "Roeschenz",
      "zip": "4095"
    }
  },
  {
    "id": 2,
    "firstname": "Max",
    "lastname": "Test",
    "birthdate": "1999-12-31T23:00:00.000Z",
    "email": "max@meier.ch",
    "address": {
      "id": 3,
      "address": "Hochwalderstrasse 23",
      "city": "Roeschenz",
      "zip": "4095"
    }
  },
  {
    "id": 4,
    "firstname": "Max",
    "lastname": "Test",
    "birthdate": "1999-12-31T23:00:00.000Z",
    "email": "max@meier.ch",
    "address": {
      "id": 3,
      "address": "Hochwalderstrasse 23",
      "city": "Roeschenz",
      "zip": "4095"
    }
  },
  {
    "id": 5,
    "firstname": "Max",
    "lastname": "Test",
    "birthdate": "1999-12-31T23:00:00.000Z",
    "email": "max@meier.ch",
    "address": {
      "id": 3,
```

Neuen Kunden mit neuer Adresse erstellen

Request: `POST /library/customers`

Erwartet: 201 Created, Kunde und Adresse werden erstellt

Response(26ms) × 21-get-customers-by-address-id.png U

```
HTTP/1.1 201
Content-Type: application/json
Content-Length: 195
Date: Mon, 20 Apr 2026 12:01:40 GMT
Connection: close

{
  "id": 6,
  "firstname": "Hans",
  "lastname": "Vettel",
  "birthdate": "2000-01-01T00:00:00.000Z",
  "email": "max@meier.ch",
  "address": {
    "id": 9,
    "address": "Lustige Strasse 123",
    "city": "Hinterhausen",
    "zip": "4095"
  }
}
```

Neuen Kunden mit bestehender Adresse erstellen

Request: `POST /library/customers`

Erwartet: 201 Created, Kunde wird mit bestehender Adresse verknüpft

Response(9ms) × 22-create-new-customer-with-new-address.png U 21

```
HTTP/1.1 201
Content-Type: application/json
Content-Length: 190
Date: Mon, 20 Apr 2026 12:02:06 GMT
Connection: close

{
  "id": 7,
  "firstname": "Max",
  "lastname": "Test",
  "birthdate": "2000-01-01T00:00:00.000Z",
  "email": "max@meier.ch",
  "address": {
    "id": 3,
    "address": "Hochwalderstrasse 23",
    "city": "Roeschenz",
    "zip": "4095"
  }
}
```

Kunden mit nicht existierender Adresse erstellen

Request: `POST /library/customers`

Erwartet: 404 Not Found, Adresse existiert nicht

Response(7ms) ×

```
HTTP/1.1 404
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 20 Apr 2026 12:02:38 GMT
Connection: close

{
  "timestamp": "2026-04-20T12:02:38.147Z",
  "status": 404,
  "error": "Not Found",
  "path": "/library/customers"
}
```

Kunden ohne Adresse erstellen

Request: `POST /library/customers`

Erwartet: 400 Bad Request, Adresse ist erforderlich

Response(4ms) 24-create-new-customer-with-non-

```
HTTP/1.1 400
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 20 Apr 2026 12:03:05 GMT
Connection: close

{
  "timestamp": "2026-04-20T12:03:05.590Z",
  "status": 400,
  "error": "Bad Request",
  "path": "/library/customers"
}
```

Kunden bearbeiten

Request: `PATCH /library/customers/1`

Erwartet: 200 OK, E-Mail wird aktualisiert

Response(14ms)

```
HTTP/1.1 200
Content-Type: application/json
Content-Length: 196
Date: Mon, 20 Apr 2026 12:04:07 GMT
Connection: close

{
  "id": 1,
  "firstname": "Max",
  "lastname": "Meier",
  "birthdate": "1999-12-31T23:00:00.000Z",
  "email": "newmail@crazy.com",
  "address": {
    "id": 3,
    "address": "Hochwalderstrasse 23",
    "city": "Roeschenz",
    "zip": "4095"
  }
}
```

Nicht existierenden Kunden bearbeiten

Request: `PATCH /library/customers/9999`

Erwartet: 404 Not Found

Response(7ms)

```
HTTP/1.1 404
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 20 Apr 2026 12:06:48 GMT
Connection: close

{
  "timestamp": "2026-04-20T12:06:48.048Z",
  "status": 404,
  "error": "Not Found",
  "path": "/library/customers/9999"
}
```

Kunden löschen

Request: `DELETE /library/customers/1`

Erwartet: 204 No Content, Kunde wird gelöscht

Response(479ms)

```
HTTP/1.1 204
Date: Mon, 20 Apr 2026 12:15:14 GMT
Connection: close
```

Verifikation: GET nach dem Löschen

```
HTTP/1.1 404
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 20 Apr 2026 12:17:52 GMT
Connection: close

{
  "timestamp": "2026-04-20T12:17:52.336Z",
  "status": 404,
  "error": "Not Found",
  "path": "/library/customers/1"
}
```

Kunden mit offener Ausleihe löschen

Request: `DELETE /library/customers/3`

Erwartet: 409 Conflict, Kunde hat noch offene Ausleihen

Response(519ms)

```
HTTP/1.1 409
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 20 Apr 2026 12:22:51 GMT
Connection: close

{
  "timestamp": "2026-04-20T12:22:51.660Z",
  "status": 409,
  "error": "Conflict",
  "path": "/library/customers/3"
}
```

Nicht existierenden Kunden löschen

Request: `DELETE /library/customers/9999`

Erwartet: 404 Not Found

```
HTTP/1.1 404
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 20 Apr 2026 12:23:17 GMT
Connection: close

{
  "timestamp": "2026-04-20T12:23:17.671Z",
  "status": 404,
  "error": "Not Found",
  "path": "/library/customers/9999"
}
```

5.4 Ausleihen

Alle Ausleihen abrufen

Request: `GET /library/borrowings`

Erwartet: 200 OK, JSON Array mit allen aktiven Ausleihen

Response(85ms)

```
HTTP/1.1 200
Content-Type: application/json
Content-Length: 397
Date: Mon, 20 Apr 2026 12:45:02 GMT
Connection: close

[
  {
    "id": 2,
    "borrowdate": "2026-04-17",
    "duration": 28,
    "customer": {
      "id": 3,
      "firstname": "Max",
      "lastname": "Meier",
      "birthdate": "1999-12-31T23:00:00.000Z",
      "email": "max@meier.ch",
      "address": {
        "id": 4,
        "address": "Hochwalderstrasse 123",
        "city": "Roeschenz",
        "zip": "4095"
      }
    },
    "media": {
      "id": 2,
      "title": "Harry Potter",
      "author": "J.K. Rowling",
      "genre": "Adventure",
      "minage": 10,
      "isbn": "978-3-16-148410-0",
      "locationcode": "A1"
    }
  }
]
```

Neue Ausleihe erstellen

Request: `POST /library/borrowings`

Erwartet: 201 Created, Ausleihe wird mit heutigem Datum erstellt

```
HTTP/1.1 201
Content-Type: application/json
Content-Length: 394
Date: Mon, 20 Apr 2026 12:45:30 GMT
Connection: close

{
  "id": 6,
  "borrowdate": "2026-04-20",
  "duration": 14,
  "customer": {
    "id": 4,
    "firstname": "Max",
    "lastname": "Test",
    "birthdate": "1999-12-31T23:00:00.000Z",
    "email": "max@meier.ch",
    "address": {
      "id": 3,
      "address": "Hochwalderstrasse 23",
      "city": "Roeschenz",
      "zip": "4095"
    }
  },
  "media": {
    "id": 5,
    "title": "Fourth Wing",
    "author": "Rebecca Yarros",
    "genre": "Romantasy",
    "minage": 10,
    "isbn": "978-3-16-148410-0",
    "locationcode": "A1"
  }
}
```

Ausleihe mit nicht existierendem Kunden erstellen

Request: `POST /library/borrowings`

Erwartet: 404 Not Found, Kunde existiert nicht

Response(21ms) 32-create-new-borrowing.png

```
HTTP/1.1 404
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 20 Apr 2026 12:46:18 GMT
Connection: close

{
  "timestamp": "2026-04-20T12:46:18.225Z",
  "status": 404,
  "error": "Not Found",
  "path": "/library/borrowings"
}
```

Ausleihe mit nicht existierendem Medium erstellen

Request: `POST /library/borrowings`

Erwartet: 404 Not Found, Medium existiert nicht

Response(10ms)

```
HTTP/1.1 404
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 20 Apr 2026 12:46:43 GMT
Connection: close

{
  "timestamp": "2026-04-20T12:46:43.475Z",
  "status": 404,
  "error": "Not Found",
  "path": "/library/borrowings"
}
```

Ausleihe mit bereits ausgeliehenem Medium erstellen

Request: `POST /library/borrowings`

Erwartet: 409 Conflict, Medium ist bereits ausgeliehen

Response(11ms)

```
HTTP/1.1 409
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 20 Apr 2026 12:47:13 GMT
Connection: close

{
  "timestamp": "2026-04-20T12:47:13.764Z",
  "status": 409,
  "error": "Conflict",
  "path": "/library/borrowings"
}
```

Ausleihe verlängern

Request: `PATCH /library/borrowings/6`

Erwartet: 200 OK, Dauer wird auf 28 Tage aktualisiert

Response(15ms)

```
HTTP/1.1 200
Content-Type: application/json
Content-Length: 394
Date: Mon, 20 Apr 2026 12:47:39 GMT
Connection: close

{
  "id": 6,
  "borrowdate": "2026-04-20",
  "duration": 28,
  "customer": {
    "id": 4,
    "firstname": "Max",
    "lastname": "Test",
    "birthdate": "1999-12-31T23:00:00.000Z",
    "email": "max@meier.ch",
    "address": {
      "id": 3,
      "address": "Hochwalderstrasse 23",
      "city": "Roeschenz",
      "zip": "4095"
    }
  },
  "media": {
    "id": 5,
    "title": "Fourth Wing",
    "author": "Rebecca Yarros",
    "genre": "Romantasy",
    "minage": 10,
    "isbn": "978-3-16-148410-0",
    "locationcode": "A1"
  }
}
```

Nicht existierende Ausleihe verlängern

Request: `PATCH /library/borrowings/9999`

Erwartet: 404 Not Found

Response(8ms) 36-edit-borrowing.png U 35-c

```
HTTP/1.1 404
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 20 Apr 2026 12:47:58 GMT
Connection: close

{
  "timestamp": "2026-04-20T12:47:58.097Z",
  "status": 404,
  "error": "Not Found",
  "path": "/library/borrowings/9999"
}
```

Ausleihe beenden

Request: `DELETE /library/borrowings?mediaId=5`

Erwartet: 204 No Content, Ausleihe wird gelöscht

Response(9ms)

```
HTTP/1.1 204
Date: Mon, 20 Apr 2026 12:48:21 GMT
Connection: close
```

Verifikation: GET nach dem Beenden

Response(8ms) 39-delete-non-existing-borrowing.png U 38-c

```
HTTP/1.1 200
Content-Type: application/json
Content-Length: 397
Date: Mon, 20 Apr 2026 12:48:58 GMT
Connection: close

[
  {
    "id": 2,
    "borrowdate": "2026-04-17",
    "duration": 28,
    "customer": {
      "id": 3,
      "firstname": "Max",
      "lastname": "Meier",
      "birthdate": "1999-12-31T23:00:00.000Z",
      "email": "max@meier.ch",
      "address": {
        "id": 4,
        "address": "Hochwalderstrasse 123",
        "city": "Roeschenz",
        "zip": "4095"
      }
    },
    "media": {
      "id": 2,
      "title": "Harry Potter",
      "author": "J.K. Rowling",
      "genre": "Adventure",
      "minage": 10,
      "isbn": "978-3-16-148410-0",
      "locationcode": "A1"
    }
  }
]
```

Ausleihe mit nicht existierender Media-ID beenden

Request: `DELETE /library/borrowings?mediaId=9999`

Erwartet: 404 Not Found

Response(6ms) × 38-delete-borrowing.png U

```
HTTP/1.1 404
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 20 Apr 2026 12:48:37 GMT
Connection: close

{
  "timestamp": "2026-04-20T12:48:37.718Z",
  "status": 404,
  "error": "Not Found",
  "path": "/library/borrowings"
}
```